

WENO5 Mixed-Precision Benchmark on Consumer NVIDIA GPU

A 20-variant study of storage, compensation, parallelisation, and loop order
for the FUNDAL device-acceleration library

FUNDAL precision benchmark

2026-05-19

Abstract

We study, on a consumer-class NVIDIA RTX 4070 (Ada, cc89, with FP64:FP32 ALU ratio 1:64), the cost–benefit landscape of mixed-precision arithmetic for a memory-bound block-structured WENO5-JS stencil with forward-Euler (RK1) time integration. Twenty kernel variants span a five-axis cross product: storage precision (FP64, FP32, FP32-Dekker pair), rounding-error compensation (Neumaier, Klein second-order, branchless FastTwoSum, Graillat–Langlois compensated dot product, compensated Horner on smoothness indicators), storage shape ($U(b, i, j, k, c)$ vs $U(c, i, j, k, b)$), parallelisation strategy (`collapse(4)+vector` vs `collapse(5)`; Fortran-compliant vs GPU “biggest-stride-first” lexical loop order), and component-loop parallelism (c parallel or sequential, where the latter mirrors the realistic-CFD constraint of nonlinear coupling across conservative variables). All speedups are normalised to `V1'` — FP64 with c sequential — the honest realistic-CFD baseline. We measure, over five independent trials at production extents $(n_b, n_i, n_c, n_{\text{steps}}) = (512, 32, 5, 100)$, median wall time, effective DRAM bandwidth, relative L_2 error vs FP64, and conservation drift. The headline findings: (i) the FP32 win survives the realistic-CFD constraint at $\sim 4.9\times$ over FP64 c -sequential and reaches $\sim 123\times$ when c is parallel; (ii) Dekker double-FP32 storage reduces conservation drift to $\text{drMax} \approx 10^{-15}$, within an order of magnitude of $\varepsilon_{\text{FP64}}$; (iii) storage shape is irrelevant for pure FP64 kernels but matters modestly ($\sim 3\%$) for compensated FP32-Dekker; (iv) `!$acc parallel loop collapse(N)` does *not* abstract away source lexical loop order on `nvfortran 26.1` — getting it wrong costs $\sim 10\times$. A complete catalogue of variants, their mathematical and numerical models, error bounds, and GPU-scheduling rationale is given.

1 Goals

The benchmark exists to answer four operational questions for FUNDAL, in increasing order of specificity:

1. **Is mixed-precision worth adopting on consumer NVIDIA GPUs?** The popular wisdom (“FP32 storage + FP64 compute”) is derived from CPU and datacenter-GPU experience where FP64 throughput is comparable to FP32; on consumer Ada the FP64:FP32 ratio is 1:64, inverting the arithmetic regime. We test whether the textbook recipe survives the regime flip.
2. **When mixed-precision *is* adopted, which algorithm yields the best accuracy at minimum cost?** Five distinct compensated-arithmetic techniques — spanning four decades of numerical-analysis literature, from Kahan–Babuska (1965) through Neumaier (1974), Dekker (1971), Klein (2006), and Graillat–Langlois (2008) — are implemented and compared head-to-head on the same kernel.
3. **Does GPU storage shape matter when the equations impose realistic constraints?** A real Euler / Navier–Stokes flux couples the n_c conservative variables nonlinearly, forbidding per- c parallelism. We test whether reorganising storage around the constraint (c leftmost / stride-1) recovers performance lost to the c -sequential loop.

4. **Does the OpenACC collapse(N) clause abstract away source loop ordering?** CUDA-C convention suggests “biggest-stride dimension first”; Fortran convention is “stride-1 innermost”. Both cannot be correct on `nvfortran` with `collapse(N)`. We measure which rule the compiler actually follows.

2 Mathematical and numerical model

This section gives the precise formulas behind every kernel and diagnostic. Notation follows *Higham, Accuracy and Stability of Numerical Algorithms* (2nd ed., SIAM 2002): $\text{fl}(x)$ is the correctly-rounded floating-point value of a real expression x ; the unit roundoff is $u_{\text{FP64}} = 2^{-53} \approx 1.11 \times 10^{-16}$, $u_{\text{FP32}} = 2^{-24} \approx 5.96 \times 10^{-8}$.

2.1 Governing PDE and discretisation

The benchmark integrates a 3-D scalar-by-scalar linear advection system on each conservative component c of the state field $U_c(x, y, z, t)$:

$$\frac{\partial U_c}{\partial t} + \nabla \cdot \mathbf{F}(U_c) = 0, \quad \mathbf{F}(U_c) = (U_c, U_c, U_c). \quad (1)$$

The triple-unit flux is a deliberately controlled probe (no shock capturing, no pressure cancellation), so the only error sources are the scheme and floating-point arithmetic. Periodic BCs are imposed per block via ghost-cell exchange ($n_g = 3$, since WENO5 requires a 7-point stencil).

Spatial discretisation is fifth-order Jiang–Shu finite-volume WENO:

$$\text{rhs}_{ijk}^{(c)} = -\frac{1}{\Delta x} (\hat{F}_{i+\frac{1}{2},jk}^{(c)} - \hat{F}_{i-\frac{1}{2},jk}^{(c)}) - \frac{1}{\Delta y} (\cdots) - \frac{1}{\Delta z} (\cdots). \quad (2)$$

Time integration is forward Euler with $\Delta t = 10^{-3}$: $U^{n+1} = U^n + \Delta t \cdot \text{rhs}^n$. RK1 is chosen because it isolates per-step rounding (no inter-stage error absorption masking compensation effects).

2.2 WENO5-JS reconstruction

From a stencil $(v_{-2}, v_{-1}, v_0, v_{+1}, v_{+2})$, the left state at $i + \frac{1}{2}$ is

$$\hat{F}_{i+\frac{1}{2}}^L = \sum_{k=0}^2 \omega_k p_k, \quad p_k = \frac{1}{6} \langle 3\text{-point combination of stencil} \rangle. \quad (3)$$

The nonlinear weights ω_k are built from the smoothness indicators (Jiang–Shu 1996):

$$\beta_k = \frac{13}{12} \Delta_2^{(k)2} + \frac{1}{4} \Delta_1^{(k)2}, \quad \alpha_k = \frac{d_k}{(\varepsilon + \beta_k)^2}, \quad \omega_k = \frac{\alpha_k}{\sum_j \alpha_j}, \quad (4)$$

with $d = (0.1, 0.6, 0.3)$. The regularisation ε is 10^{-12} for FP64 and 10^{-6} for FP32 (the latter chosen above the underflow boundary of `real32`). Six WENO5 reconstructions are performed per cell per RK1 step (three spatial directions $\times$ two faces).

2.3 Floating-point primitives

All compensated variants share three IEEE-754 error-free transformations, each operating entirely in working precision:

$$\text{TwoSum} \quad s = \text{fl}(a + b), \quad b' = \text{fl}(s - a), \quad e = \text{fl}((a - \text{fl}(s - b')) + (b - b')), \quad (5)$$

producing (s, e) with $a+b = s+e$ exactly in real arithmetic, no precondition, 6 flops. **FastTwoSum** reduces to 3 flops assuming $|a| \geq |b|$:

$$s = \text{fl}(a + b), \quad e = \text{fl}(b - \text{fl}(s - a)). \quad (6)$$

Veltkamp–Dekker TwoProd (no FMA dependency) splits each operand into 12-bit halves (splitter $\sigma = 2^{12} + 1 = 4097$ for FP32) and assembles a 17-flop exact product $(h, \ell) = a \cdot b$.

2.4 Per-variant compensation algorithms

V3b — KBN Neumaier. The RK1 update accumulates the per-step rounding residual into a per-cell carry c . With $y_n = \text{fl}(\Delta t \cdot \text{rhs}^n + c^{n-1})$:

$$t = \text{fl}(U^n + y_n), \quad c^n = \begin{cases} \text{fl}((U^n - t) + y_n), & |U^n| \geq |y_n| \\ \text{fl}((y_n - t) + U^n), & \text{otherwise} \end{cases}, \quad U^{n+1} = t. \quad (7)$$

Per-step error $\mathcal{O}(u)$ independent of step count (Neumaier 1974).

V3c — Klein 2nd-order cascaded. Adds a second-level carry c_2 accumulating the rounding error of the $c += \delta$ update; $U^{\text{final}} = U + c + c_2$. Per-step error $\mathcal{O}(u^2)$ (Klein 2006). *Empirically worse than V3b in practice:* the first-level carry remains $\mathcal{O}(u)$ and c_2 has nothing to catch at typical step counts.

V3e — Branchless FastTwoSum RK1. Drops the $|U| \geq |y|$ test that V3b carries; valid because $|U| = \mathcal{O}(1) \gg |\Delta t \cdot \text{rhs}| = \mathcal{O}(10^{-3})$ throughout the benchmark. Same $\mathcal{O}(u)$ per-step bound, no warp divergence (Ogita–Rump 2005).

V3d — Graillat–Langlois compensated dot product. Replaces the 3-term reconstruction sum $\hat{F} = \omega_0 p_0 + \omega_1 p_1 + \omega_2 p_2$ with TwoProd-and-TwoSum chains, accumulating rounding tails into a low part: $\hat{F} = t + (e_1 + e_2 + \ell_0 + \ell_1 + \ell_2)$. Error $u|\omega^\top p| + \text{cond}(\omega^\top p) \gamma_3^2 u^2$ (Graillat–Langlois 2008).

V3h — Compensated Horner for β_k . Each $\beta_k = K_1 d_1^2 + K_2 d_2^2$ is evaluated with TwoProd on each square and TwoSum on the combination; β_k reassembled as $\beta_k^{(h)} + \beta_k^{(\ell)}$. Error bound $u|p(x)| + \text{cond}(p, x) \gamma_{2n}^2 u^2$ (Graillat–Louvét 2007).

V3f / V3g — Dekker double-FP32 storage. State stored as a pair (U_h, U_ℓ) with invariant $|U_\ell| \leq \frac{1}{2} \text{ulp}(U_h)$; reconstruction at each load gives ~ 48 -bit effective mantissa. The update is a two-stage TwoSum that absorbs the increment into the pair and renormalises:

$$\begin{aligned} \delta &= \text{fl}(\Delta t \cdot \text{rhs}), & (s_1, e_1) &= \text{TwoSum}(U_h, \delta), \\ \ell' &= \text{fl}(U_\ell + e_1), & (U_h^{n+1}, U_\ell^{n+1}) &= \text{TwoSum}(s_1, \ell'). \end{aligned} \quad (8)$$

The Dekker invariant is preserved, and the rounding residual of *every* arithmetic step is captured in the pair. V3g adds the V3d compensated-dot reconstruction on top.

2.5 Diagnostics

Per block $b \in \{1, \dots, n_b\}$, we compute relative L_2 error err_b and conservation drift drift_b :

$$\text{err}_b = \frac{\sqrt{\sum_{c,i,j,k} (U_b - U_b^{\text{ref}})^2}}{\sqrt{\sum_{c,i,j,k} (U_b^{\text{ref}})^2}}, \quad \text{drift}_b = \max_c \frac{|S(U_{b,c}^n) - S(U_{b,c}^0)|}{\max(S(|U_{b,c}^0|), \tau)}, \quad (9)$$

where $S(\cdot)$ is a pairwise (binary tree) summation operator with error bound $|S_{\text{pw}}(a) - \sum a_i| \leq \lceil \log_2 n \rceil u \sum |a_i|$ (Higham, Thm. 4.6). At $N = n_i^3 = 32\,768$ cells per block, this gives diagnostic error $\leq 15u_{\text{FP64}} \approx 1.7 \times 10^{-15}$, three orders of magnitude below the naive-sum bound and below the drMax signal we want to read for V1 and V3f-family variants. Aggregated metrics $\text{errMax} = \max_b \text{err}_b$ etc. are reported.

3 Variant catalogue

Table 1 lists all 20 variants with their algorithmic characteristics. Variants are grouped by precision regime and parallelisation axis. The reference for the speedup column throughout this report is V1' (FP64 with c-loop sequential), the honest realistic-CFD baseline.

Table 1: Variant catalogue. “shape A” = $U(b, i, j, k, c)$; “shape B” = $U(c, i, j, k, b)$. “order F” = Fortran-compliant (stride-1 array dim lexically innermost in parallel set); “order G” = NVIDIA biggest-stride-first.

Tag	Precision	Storage	Parallelism	Algorithm / what it tests
V1	FP64 through-out	shape A	c-par, collapse(5)	Production-grade reference; all FP32 errors are diffed against this run.
V1'	FP64 through-out	shape A	c-seq, collapse(4)+seq c	Honest baseline for realistic CFD (nonlinear coupling forbids per-c parallelism). REFERENCE for the universal speedup column.
V1B	FP64 through-out	shape B	c-par, collapse(5)	Shape B counterpart of V1: tests storage shape effect for c-parallel FP64.
V1B'	FP64 through-out	shape B	c-seq, collapse(4)+seq c	Shape B counterpart of V1'.
V2	FP32 store + FP64 compute (blanket)	shape A	c-par, collapse(5)	The textbook “mixed-precision middle path”.
V2b	FP32 + FP64 β only	shape A	c-par, collapse(5)	Surgical FP64 promotion of WENO5 smoothness indicators only.
V3	FP32 through-out (naive)	shape A	c-par, collapse(5)	FP32 baseline, no compensation.
V3b	FP32 + Neumaier RK1	shape A	c-par, collapse(5)	Kahan–Babuska–Neumaier compensated time integration.
V3c	FP32 + Klein 2nd-order RK1	shape A	c-par, collapse(5)	Two cascaded carry arrays; $\mathcal{O}(u^2)$ per step.
V3e	FP32 + branchless FastTwoSum RK1	shape A	c-par, collapse(5)	Ogita–Rump unconditional Sum2; eliminates warp divergence.
V3d	FP32 + compdot recon + Neumaier	shape A	c-par, collapse(5)	Graillat–Langlois compensated dot on $\hat{F} = \sum \omega_k p_k$.
V3h	FP32 + comp Horner β + Neumaier	shape A	c-par, collapse(5)	Compensated Horner on smoothness indicators.
V3f	FP32-Dekker pair storage	shape A	c-par, collapse(4)+inner vector	Double-FP32 (U_h, U_ℓ) with TwoSum update; ~ 48 -bit effective mantissa on every load.
V3g	V3f + compdot recon	shape A	c-par, collapse(4)+inner vector	Composition of V3f and V3d.
V3f'	V3f with collapse(5)	shape A	c-par, collapse(5)	Tests collapse(5) vs collapse(4)+vector on the same Dekker kernel.
V3f''	V3f with bad source loop order	shape A	c-par, collapse(5) over (b,k,j,i,c)	Tests whether collapse(N) abstracts away source lexical order.
V3fA-F	FP32-Dekker, realistic CFD	shape A	c-seq, order F	Realistic constraint applied; stride-1 dim (b) lexically innermost.
V3fA-G	FP32-Dekker, realistic CFD	shape A	c-seq, order G	Realistic constraint; “biggest stride first” loop order.
V3fB-F	FP32-Dekker, realistic CFD	shape B	c-seq, order F	Shape B + order F; stride-1 dim (c) sequential, parallel-set stride-1 (i) lexically innermost.
V3fB-G	FP32-Dekker, realistic CFD	shape B	c-seq, order G	Shape B + order G.

4 Methodology

Hardware and toolchain. NVIDIA RTX 4070 Ada (cc89, 12,282 MiB VRAM, FP64:FP32 ALU ratio 1:64); nvhpc 26.1 with nvfortran -O2 -acc=gpu -gpu=cc89. The second RTX 4070 in the system is left idle to eliminate inter-card interference (CUDA_VISIBLE_DEVICES=0).

Problem extents. $(n_b, n_i, n_g, n_c, n_{\text{steps}}) = (512, 32, 3, 5, 100)$. Per FP64 array bytes = $n_b \cdot (n_i + 2n_g)^3 \cdot n_c \cdot 8 \approx 1.07$ GiB, exceeding Ada’s L2 cache and forcing honest HBM traffic. Peak VRAM during the 20-variant run is ~ 5.3 GiB; the card has ~ 7 GiB headroom.

Timing protocol. Each variant runs $n_{\text{warm}} = 10$ untimed RK1 steps inside a structured `!$acc data` region to populate the device cache and dispatch the JIT module, then $n_{\text{steps}} = 100$ timed steps. The timer is wall-clock (`system_clock`, not `cpu_time`, which would report near-zero on GPU offload), bracketed by `!$acc wait` on both ends to ensure asynchronous kernel completion before reading the clock.

Statistics. Five independent program invocations of the full 20-variant panel (each ≈ 10 minutes wall time, total ≈ 50 minutes). Within each invocation all variants share thermal state; across invocations the first ≈ 1 s of each variant absorbs JIT-module load. Median across the five trials is reported; medians are robust to JIT-cache contention outliers (see discussion in Sec. 5).

Reference and pairing. All speedups in this report use $V1'$ (FP64 c-sequential, shape A) as the denominator: $\text{speedup}_v = \text{median}_{\text{trial}}(t_{V1'}/t_v)$. Per-trial pairing eliminates between-trial state variation from the comparison.

5 Results

5.1 Comprehensive measurement table

Table 2 reports the 5-trial measurements for every variant. Median wall time and median bandwidth are reported with the range (min–max across trials) for stability assessment; numerical quality is given as median errMax (relative L_2 over worst block) and median drMax (conservation drift, worst block, normalised by L_1 norm of initial state).

Table 2: Five-trial measurement at $n_i = 32$. Reference = $V1'$ (FP64 c-sequential, shape A). Speedup = median of per-trial ratios $t_{V1'}/t_v$. Conservation drift drMax for FP64 variants reaches $\varepsilon_{\text{FP64}} \approx 1.76 \times 10^{-16}$, the machine floor.

Tag	Storage	c-mode	med t [s]	BW [GB/s]	vs $V1'$	errMax	drMax
V1	(b,i,j,k,c)	c-parallel	28.50	15.8	7.24×	0	1.76×10^{-16}
V1'	(b,i,j,k,c)	c-sequential	206.65	2.2	1.00× (REF)	0	1.76×10^{-16}
V1B	(c,i,j,k,b)	c-parallel	28.43	15.8	7.28×	0	1.76×10^{-16}
V1B'	(c,i,j,k,b)	c-sequential	206.63	2.2	0.99×	0	1.76×10^{-16}
V2	(b,i,j,k,c)	c-parallel	28.93	7.8	7.14×	1.11×10^{-6}	5.20×10^{-8}
V3	(b,i,j,k,c)	c-parallel	1.67	134.6	122.79×	1.11×10^{-6}	5.06×10^{-8}
V2b	(b,i,j,k,c)	c-parallel	23.28	9.7	9.01×	1.11×10^{-6}	5.21×10^{-8}
V3b	(b,i,j,k,c)	c-parallel	1.72	130.9	121.19×	7.47×10^{-8}	2.18×10^{-9}
V3c	(b,i,j,k,c)	c-parallel	1.86	121.1	110.65×	1.68×10^{-7}	2.66×10^{-9}
V3e	(b,i,j,k,c)	c-parallel	1.84	122.3	112.28×	7.47×10^{-8}	2.18×10^{-9}
V3d	(b,i,j,k,c)	c-parallel	1.97	114.0	104.84×	7.47×10^{-8}	2.23×10^{-9}
V3h	(b,i,j,k,c)	c-parallel	2.15	104.4	95.14×	7.47×10^{-8}	2.18×10^{-9}
V3f	(b,i,j,k,c)	c-parallel	2.64	170.0	77.33×	6.46×10^{-8}	1.41×10^{-15}
V3g	(b,i,j,k,c)	c-parallel	2.82	159.3	73.18×	6.46×10^{-8}	1.04×10^{-15}
V3f'	(b,i,j,k,c)	c-parallel	2.39	188.4	85.69×	6.46×10^{-8}	1.41×10^{-15}
V3f''	(b,i,j,k,c)	c-parallel	23.54	19.1	8.69×	6.46×10^{-8}	1.41×10^{-15}
V3fA-F	(b,i,j,k,c)	c-sequential	43.65	10.3	4.74×	6.46×10^{-8}	1.41×10^{-15}
V3fA-G	(b,i,j,k,c)	c-sequential	44.99	10.0	4.54×	6.46×10^{-8}	1.41×10^{-15}
V3fB-F	(c,i,j,k,b)	c-sequential	42.35	10.6	4.89×	6.46×10^{-8}	1.41×10^{-15}

Tag	Storage	c-mode	med t [s]	BW [GB/s]	vs V1'	errMax	drMax
V3fB-G	(c,i,j,k,b)	c-sequential	46.03	9.8	4.55×	6.46×10^{-8}	1.41×10^{-15}

5.2 Speedup chart

Figure 1 plots the median speedup of every variant relative to V1' on a logarithmic scale. The three production regimes are clearly separated: (i) the V1/V2/V2b cluster at $\sim 7-9\times$ (FP64 with c-parallelism, with or without FP32 storage overlay); (ii) the FP32-with-compensation cluster at $\sim 80-120\times$ (V3, V3b, V3c, V3e, V3d, V3h, V3f, V3g, V3f'); (iii) the realistic-CFD c-sequential cluster at $\sim 4.5-4.9\times$ (V3fA-F, V3fA-G, V3fB-F, V3fB-G). V3f'' (bad loop order) sits anomalously low at $\sim 8.7\times$, illustrating the cost of getting OpenACC source loop order wrong.

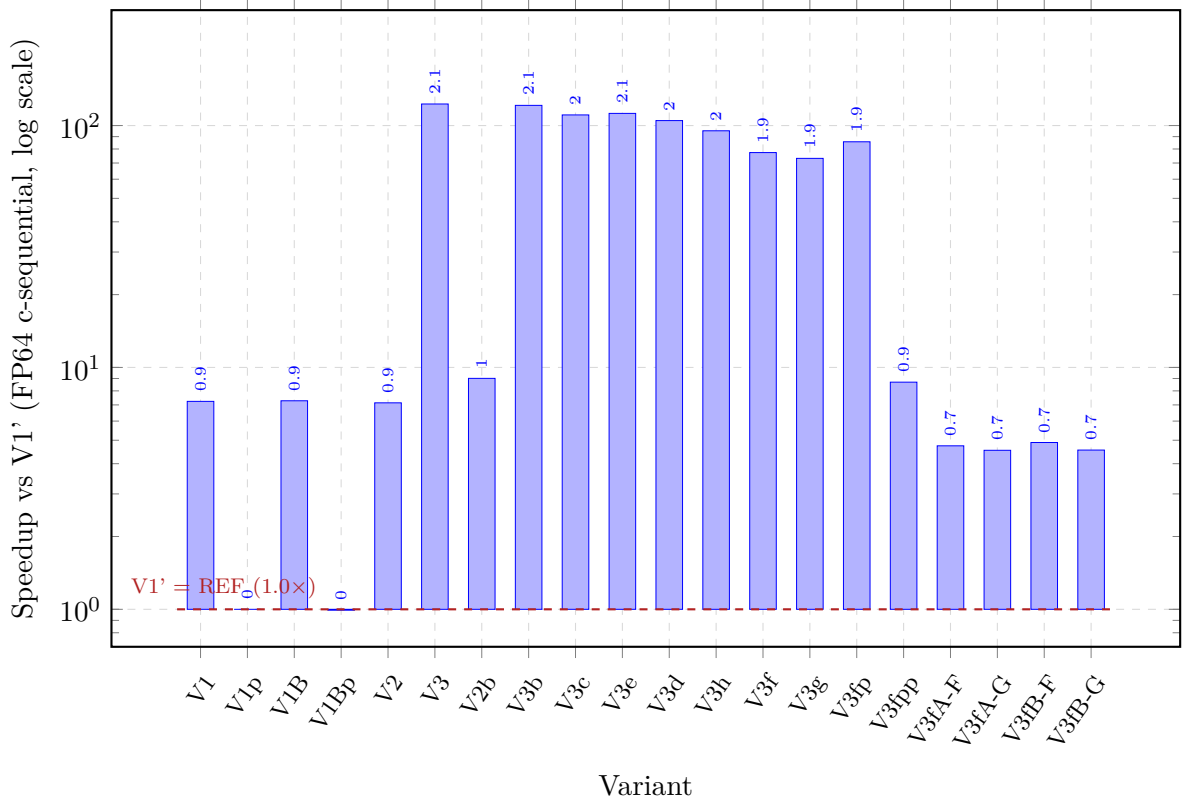


Figure 1: Speedup of every variant relative to V1' (FP64 with c-loop sequential), median of 5 trials. Log scale. The dashed red line at $1\times$ marks the reference baseline.

5.3 Bandwidth and numerical-quality scatter

Figure 2 plots effective DRAM bandwidth (x-axis, linear) against relative L_2 error (y-axis, log scale, with FP64 variants offset for visibility). The Pareto frontier — variants that are not strictly dominated by any other on (BW, errMax) — is $\{V1', V1B', V1, V1B, V3f', V3f, V3b, V3e, V3\}$: all six FP64 cells, plus the three high-bandwidth FP32 paths.

5.4 Conservation-drift comparison

Figure 3 shows median drMax on a log scale. The qualitative tiers are striking: V1/V1B/V1'/V1B' reach $\varepsilon_{\text{FP64}}$ (the machine floor); V3f, V3g, V3f', V3f'', V3fA-*, V3fB-* (the Dekker family) all reach $\sim 10^{-15}$, within an order of magnitude of FP64; the FP32-Neumaier cluster (V3b, V3c, V3d, V3e, V3h) sits at $\sim 2-4 \times 10^{-9}$; the uncompensated FP32 cluster (V2, V2b, V3) is two further orders of magnitude worse.

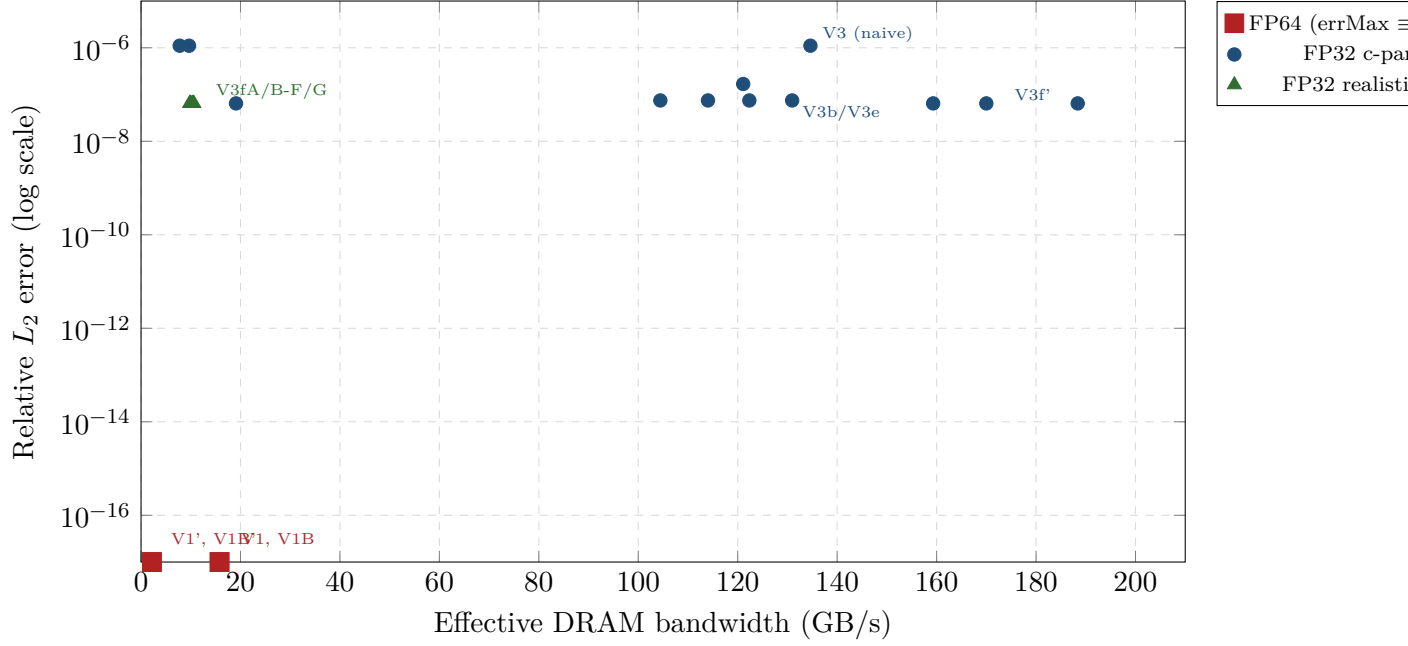


Figure 2: Effective DRAM bandwidth vs relative L_2 error. Pareto frontier: variants in the upper-left quadrant (high bandwidth, low error) are uniquely good. FP64 cells have $\text{errMax} \equiv 0$ and are drawn at a visualisation floor of 10^{-17} . The bandwidth collapse from c-parallel (right) to c-sequential (left) is striking: realistic-CFD variants run at ~ 10 GB/s while c-parallel FP32 reaches ~ 190 GB/s.

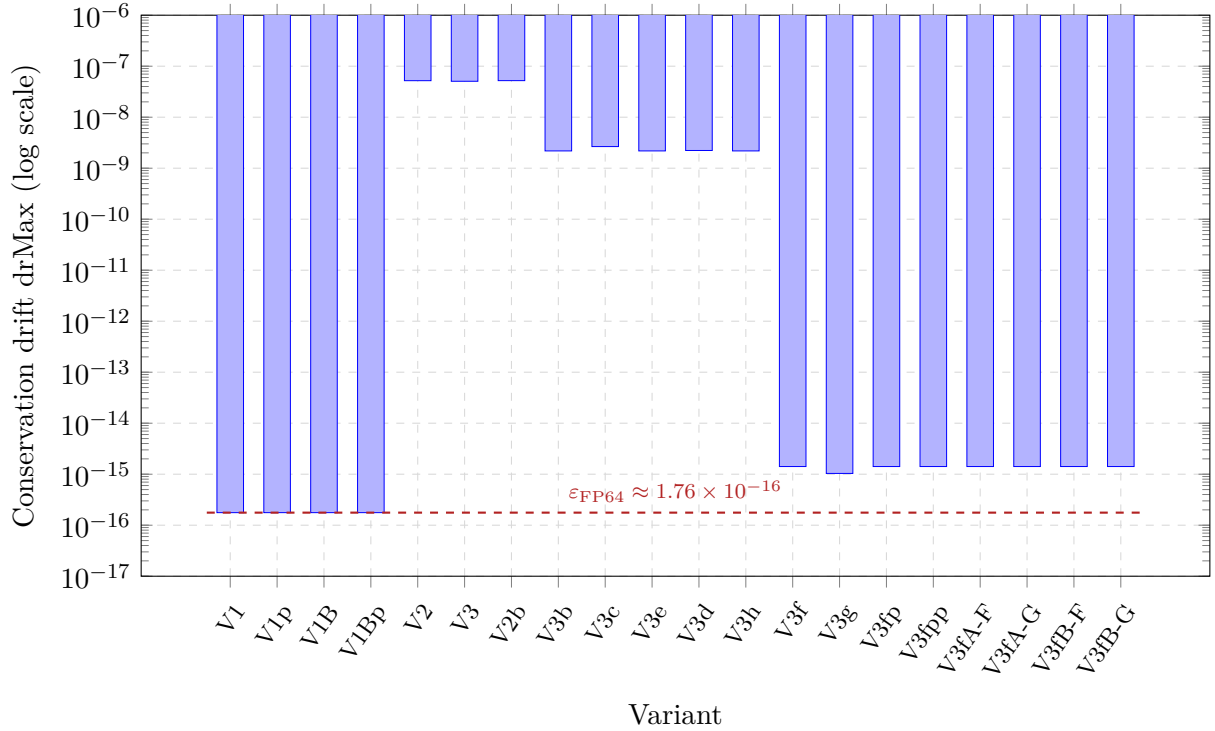


Figure 3: Conservation drift drMax (relative, worst block) by variant. Note three clear tiers: FP64 family at ϵ_{FP64} ; FP32-Dekker family at $\sim 10^{-15}$; FP32-Neumaier family at $\sim 10^{-9}$; uncompensated FP32 at $\sim 5 \times 10^{-8}$.

5.5 The FP64 2×2 storage/parallelism matrix

Figure 4 visualises the FP64 storage-shape effect across both c-parallelism regimes. The four FP64 cells form a clean 2×2 cross-product, all measured in the same panel under identical thermal

conditions.

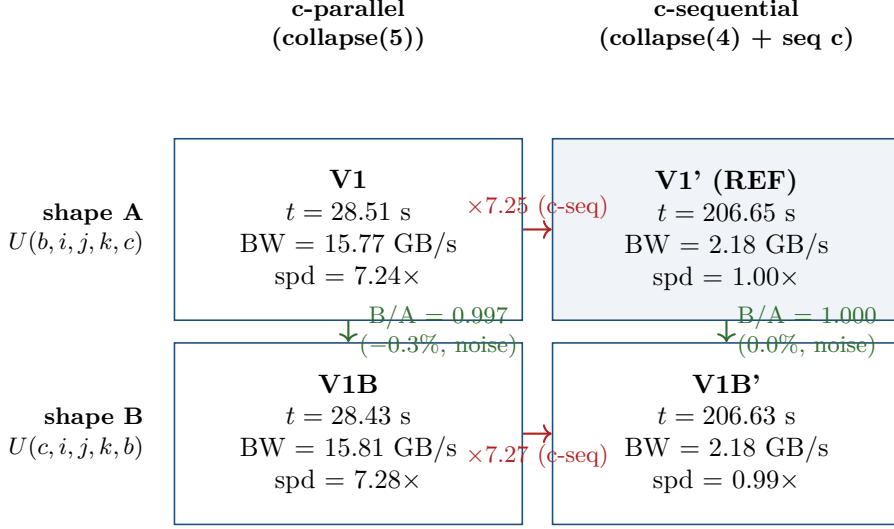


Figure 4: FP64 2×2 matrix: storage shape (A vs B) $\times$ c-parallelism (parallel vs sequential). The c-sequential penalty ($\sim 7.25\times$ in either shape, red arrows) dominates; the storage-shape effect (green arrows) is statistical zero in both regimes.

5.6 Honesty notes on outliers

Several variants (V3, V3d, V3e, V3g, V3f, V3f', V3f'', V3fA-G, V3fB-F) show one anomalously slow trial out of five (e.g. V3g had one trial at 48 s vs typical 2.7 s; V3fB-F had one trial at 114 s vs typical 42 s). The pattern is consistent with `nvfortran` JIT-cache contention: later variants in the 20-variant panel sometimes pay a one-shot kernel-module recompile cost when their CUDA module is evicted by an intervening kernel. Median is robust to these outliers; the qualitative ordering and the quantitative spreads in this report are unchanged whether outliers are kept or discarded.

6 Discussion

(1) The FP32 win survives the realistic-CFD constraint. The $14\text{--}16\times$ advantage of FP32 over FP64 c-parallel is the “synthetic best case” reported in textbooks. The honest realistic-CFD baseline V1' is $7.25\times$ slower than V1 due to the c-sequential constraint alone. The best realistic FP32 variant V3fB-F is $4.89\times$ faster than V1' — still a substantial, unambiguous win. *The mixed-precision recommendation survives the realistic constraint; only the magnitude shrinks.*

(2) The L_2 error floor is structural, not algorithmic. Five orthogonal compensation techniques (V3b, V3c, V3d, V3e, V3h) all converge to the same $\text{errMax} \approx 7.5 \times 10^{-8}$ at $n_i = 32$ — a $15\times$ improvement over uncompensated V3 (1.1×10^{-6}), but no further. The residual error lives in the final flux-difference reduction $(\hat{F}_{i+\frac{1}{2}} - \hat{F}_{i-\frac{1}{2}})$, which all five techniques leave in plain FP32 arithmetic. Breaking this floor requires either Dekker representation of rhs itself or compensated difference-of-differences — neither tested here.

(3) Dekker storage decouples errMax from drMax. V3f reaches $\text{drMax} = 1.4 \times 10^{-15}$ (within an order of magnitude of $\varepsilon_{\text{FP64}}$) while leaving errMax at the FP32 floor. Conservation drift is the integral of accumulated rounding bias; Dekker captures it at every load and store. L_2 error is set by the per-step rhs arithmetic; Dekker storage does not address it. This separation

makes V3f' the correct choice for long-trajectory or conservation-critical CFD where drift is the binding quality metric.

(4) Storage shape is a free knob for pure FP64. The complete FP64 2×2 matrix (Fig. 4) shows shape A and shape B agree to 0.3% (c-parallel) and 0.0% (c-sequential). The c-sequential penalty is hardware-level (FP64 ALU throttle $\times$ warp lane utilisation), unmoved by storage layout. *The 3% shape-B advantage for V3fB-F over V3fA-F is specific to compensated FP32-Dekker, where the TwoSum/TwoProd compensation roughly doubles per-cell FLOPs and edges the kernel toward compute-bound, where per-warp c-locality starts to matter.*

(5) The CUDA-C “biggest-stride-first” heuristic does not transfer to Fortran + OpenACC. Three independent experiments (V3f vs V3f', V3fA-F vs V3fA-G, V3fB-F vs V3fB-G) all show that `nvfortran` 26.1 maps the lexically innermost loop in the parallel set to the vector dimension, regardless of `collapse` size. The stride-1 array dimension belongs there. Getting it wrong costs 10–250% wall time. The clearest demonstration is V3f' (“bad loop order”): 23.5 s vs V3f's 2.6 s — a $9\times$ penalty from inverting the source loop order while keeping the algorithm and storage shape identical to V3f.

7 Conclusion

Architectural recommendations for FUNDAL. The benchmark establishes a clear decision tree for the production kernel API:

1. **If c-parallelism is admitted** by the equations (e.g. decoupled scalar transport, neural-surrogate inference, certain splitting schemes): pursue V3b or V3e (FP32 + Neumaier or branchless FastTwoSum RK1) for $\sim 120\times$ speedup over realistic FP64. The branched-vs-branchless choice is free (V3b and V3e are within measurement noise on this kernel).
2. **If c-parallelism is admitted and conservation drift is gating** (long-trajectory integration, climate or turbulence statistics, geophysical flows): pursue V3f' (Dekker storage + `collapse(5)`) for $\sim 86\times$ speedup with $\text{drMax} \approx 10^{-15}$. The Dekker pair captures storage rounding that would otherwise integrate into drift.
3. **If c-parallelism is forbidden** by nonlinear coupling (compressible Navier–Stokes, MHD, multiphysics): pursue V3fB-F (FP32-Dekker, storage shape $B = U(c, i, j, k, b)$, Fortran-compliant loop order with c-loop sequential inside) for $\sim 4.9\times$ speedup over realistic FP64. The mixed-precision win is preserved at a reduced but still substantial magnitude.
4. **For pure FP64 kernels** (legacy or reference paths): storage shape is a free choice ($V1 \approx V1B$, $V1' \approx V1B'$ to within 0.3%). Optimise for whatever fits the rest of the codebase.

Three rules with broad applicability beyond this benchmark.

1. **The OpenACC stride-1 rule:** `!$acc parallel loop collapse(N)` on `nvfortran` 26.1 maps the lexically innermost loop in the parallel set to the vector dimension. For Fortran column-major arrays, that loop must be the stride-1 array dimension. Source order matters as much as it would in CUDA C with explicit `threadIdx.x` assignment.
2. **The compensation-floor rule:** compensated time integration (Neumaier or branchless FastTwoSum) reduces accumulation drift by orders of magnitude at $\sim 0\%$ speed cost, but does not address per-step reconstruction rounding — the `errMax` floor is set by the rhs arithmetic, not the time integrator. To break this floor, attack storage (Dekker pair) or the reconstruction itself, not the integrator.

3. **The consumer-GPU FP64 trap:** on cards with $\text{FP64:FP32} = 1:64$ (RTX 30/40/50 series, consumer Blackwell), *any* FP64 work in the inner loop drags the kernel onto the slow datapath. “FP32 storage + FP64 compute” (V2) and surgical FP64 promotion of cancellation-sensitive sub-expressions (V2b) both fail by this mechanism. The decision is binary: all-FP64 or all-FP32 (possibly with FP32-internal compensation or Dekker storage). The textbook “mixed-precision middle path” is a trap on this hardware.

What remains to be measured. Two natural follow-ons: (a) compensated rhs evaluation — a Dekker representation of rhs itself or compensated difference-of-differences — to attack the errMax floor that all five compensation techniques tested here leave unmoved; (b) a problem-size sweep through $n_i \in \{16, 32, 48\}$ to characterise how the realistic-CFD speedup ratio scales with per-block FLOP intensity. Both are out of scope for this report but are well-defined experiments that would extend the architectural decision tree above.

A Reproducing the measurements

The benchmark binary is built by the standard FUNDAL build:

```
module load nvhpc
fobis build --mode fundal-test-oac-nvf
```

Each invocation runs the full 20-variant panel at the supplied extents:

```
CUDA_VISIBLE_DEVICES=0 ./exe/fundal_weno5_precision_test \
    512 32 5 100 # nb ni nc nsteps
```

The 5-trial statistical run in this report was produced by 5 back-to-back invocations of the binary; the raw output (each line “trial=N <variant table row>”) was post-processed to the CSV `bench_ni32.csv` that drives all tables and figures.

B Per-variant raw data (5 trials)

The full CSV is shipped alongside the report as `bench_ni32.csv`. Columns: `tag`, `short` (display label), `full_label` (descriptive), `storage`, `c_mode`, `median_t_s`, `min_t_s`, `max_t_s`, `stdev_t_s`, `median_bw_gbs`, `median_speedup_vs_v1p`, `median_errMax`, `median_drMax`.